

BÁO CÁO THỰC HÀNH TUẦN 3

DESIGN PATTERN

(STATE- STRATEGY – DECORATOR – COMPOSITE – OBSERVER -ADAPTER)

Nguyễn Thị Hà Như – 22635201

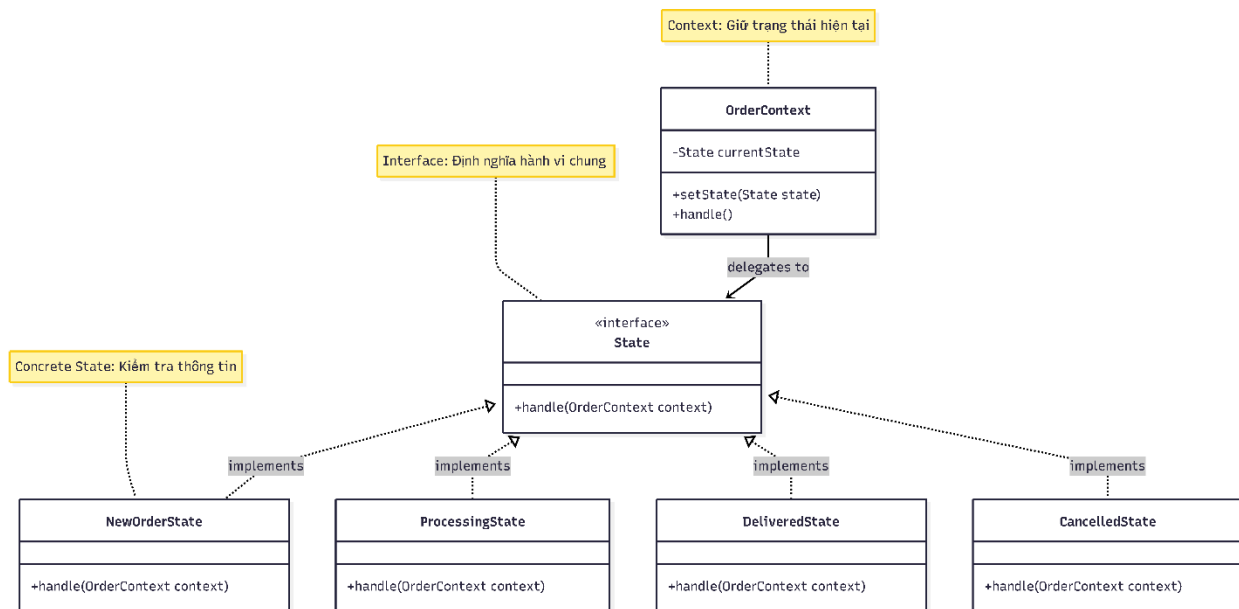
Mục lục:

Bài 1: State Pattern	1
Bài 2: Strategy Pattern	3
Bài 3: Decorator	4
Bài 4: Composite	7
Bài 5: Observer Design Pattern	9
Bài 6 : Adapter Pattern	11
Bài Tổng Hợp 6 pattern: Quản lý thư viện	13

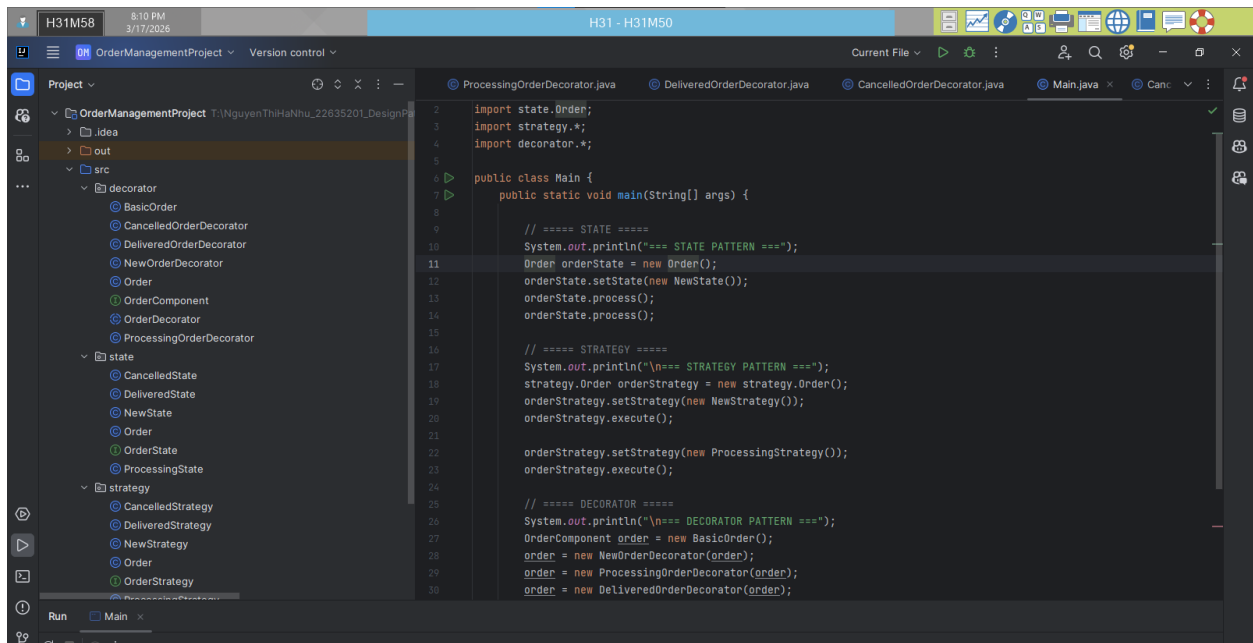
Bài 1: State Pattern

Tạo một hệ thống quản lý đơn hàng có các trạng thái như: Mới tạo, Đang xử lý, Đã giao, và Hủy.

1. Sơ đồ Design



2. Cấu trúc dự án



Kết quả

```
C:\Java\jdk-17\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2023.2\lib\idea_rt.jar=50765:C:\Program Fi
=== STATE PATTERN ===
Kiểm tra thông tin đơn hàng
Đóng gói và vận chuyển

=== STRATEGY PATTERN ===
Kiểm tra thông tin đơn hàng
Đóng gói và vận chuyển

=== DECORATOR PATTERN ===
Xử lý đơn hàng cơ bản
Kiểm tra thông tin đơn hàng
Đóng gói và vận chuyển
Đã giao hàng

Process finished with exit code 0
|
```

3. Kết luận và Đánh giá:

- State: Đây là lựa chọn tốt nhất vì bài toán yêu cầu xử lý các giai đoạn (Mới tạo -> Đang xử lý -> Đã giao). Nó giúp loại bỏ các câu lệnh if-else phức tạp và tách biệt logic của từng trạng thái. Vì bản chất của đơn hàng là một cỗ máy trạng thái (Finite State Machine). State Pattern giúp quản lý việc chuyển đổi giữa các trạng thái một cách logic, ngăn chặn việc một đơn hàng "Đã giao" lại quay ngược về "Mới tạo".
- **Mục đích:** Cho phép đối tượng thay đổi hành vi khi **trạng thái nội tại** của nó thay đổi.

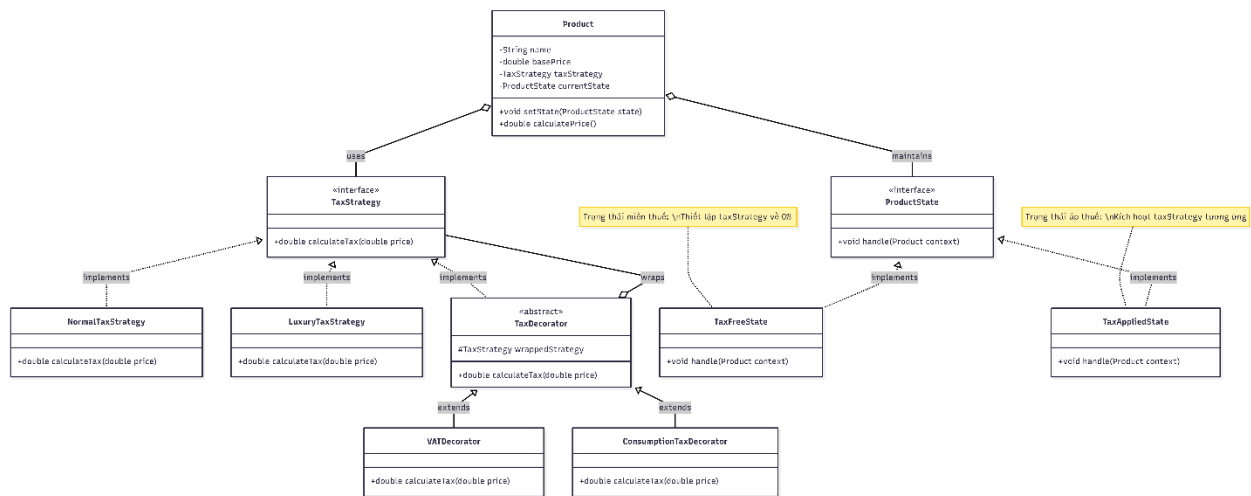
- **Cơ chế:** Đóng gói mỗi trạng thái vào một lớp riêng biệt, giúp loại bỏ logic kiểm tra điều kiện dày đặc.

- **Ứng dụng:** Quản lý vòng đời đơn hàng (Mới tạo -> Đang xử lý -> Hoàn tất) hoặc luồng giao dịch thanh toán.

Bài 2: Strategy Pattern

Tính toán thuế cho sản phẩm

1. Sơ đồ Design

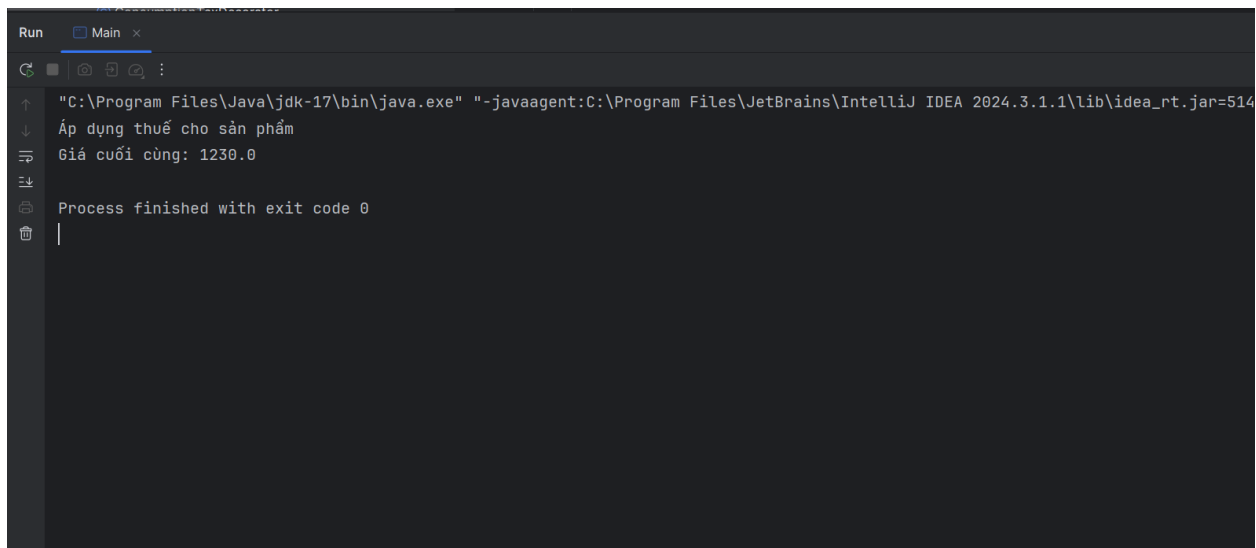


2. Cấu trúc dự án

The screenshot shows an IDE with a project named "tax-system". The project structure on the left includes folders for "decorator", "model", "state", and "strategy", each containing relevant classes. The "Main.java" file is open, showing the following code:

```
1 import ...
2
3
4
5
6 public class Main {
7     public static void main(String[] args) {
8
9         // ===== Strategy cơ bản =====
10        TaxStrategy normal = new NormalTaxStrategy();
11
12        // ===== Decorator thêm thuế =====
13        TaxStrategy withVAT = new VATDecorator(normal);
14        TaxStrategy fullTax = new ConsumptionTaxDecorator(withVAT);
15
16        // ===== Product =====
17        Product product = new Product("Laptop", basePrice: 1000, fullTax);
18
19        // ===== State =====
20        product.setState(new TaxAppliedState());
21
22        double finalPrice = product.calculatePrice();
23
24        System.out.println("Giá cuối cùng: " + finalPrice);
25    }
26 }
```

Kết quả



```
Run Main x
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\lib\idea_rt.jar=514
↓
Áp dụng thuế cho sản phẩm
Giá cuối cùng: 1230.0
Process finished with exit code 0
```

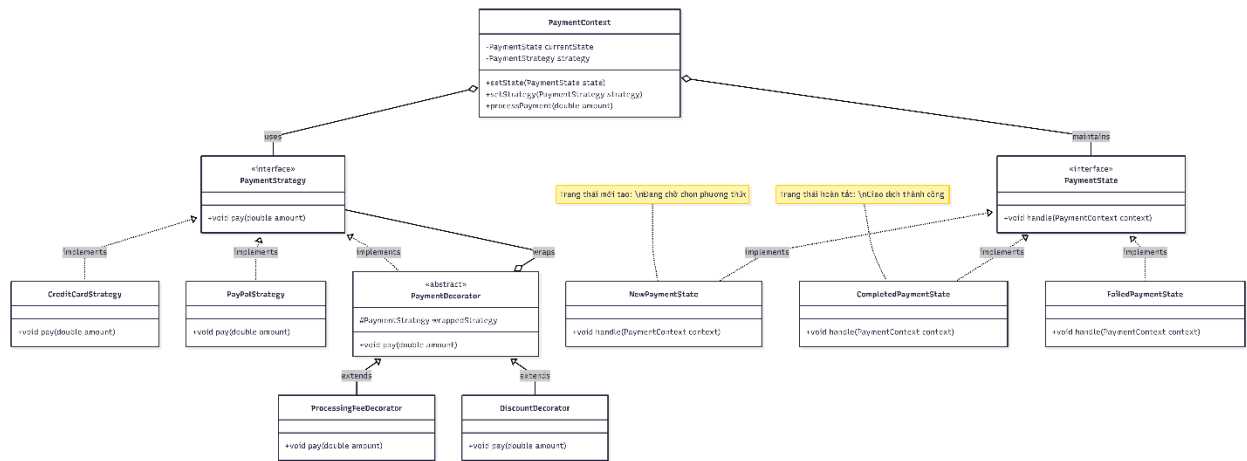
3. Kết luận

- Trong bài toán tính thuế sản phẩm, Strategy Pattern là lựa chọn phù hợp nhất vì cho phép linh hoạt thay đổi các thuật toán tính thuế.
- Trong trường hợp một sản phẩm chịu nhiều loại thuế cùng lúc, có thể kết hợp thêm
- Decorator Pattern để mở rộng chức năng mà không làm thay đổi cấu trúc ban đầu.
- State Pattern không thực sự cần thiết vì bài toán không có sự chuyển đổi trạng thái phức tạp.
- **Mục đích:** Tách biệt phần **thuật toán** ra khỏi đối tượng sử dụng, giúp thay đổi hành vi linh hoạt tại thời điểm chạy (runtime).
- **Cơ chế:** Định nghĩa một tập hợp các thuật toán (như cách tính thuế, phương thức thanh toán) trong các lớp riêng biệt cùng thực thi một interface.
- **Ứng dụng:** Giảm thiểu các câu lệnh if-else phức tạp và dễ dàng thêm thuật toán mới.

Bài 3: Decorator

Tạo một hệ thống thanh toán, nơi bạn có thể thanh toán bằng các phương thức khác nhau như Thẻ tín dụng, PayPal, và có thể thêm các phương thức thanh toán này với các tính năng bổ sung như Phí xử lý, Mã giảm giá.

1. Sơ đồ Design



2. Cấu trúc mã nguồn

The screenshot shows the project structure and the **Main.java** file. The project structure on the left includes the following packages and classes:

- context**
 - PaymentContext**
- decorator**
 - DiscountDecorator**
 - PaymentDecorator**
 - ProcessingFeeDecorator**
- state**
 - CompletedPaymentState**
 - FailedPaymentState**
 - NewPaymentState**
 - PaymentState**
- strategy**
 - CreditCardStrategy**
 - PaymentStrategy**
 - PayPalStrategy**

The **Main.java** file contains the following code:

```

6 public class Main {
7     public static void main(String[] args) {
14         // ===== STRATEGY =====
15         PaymentStrategy strategy = new CreditCardStrategy();
16
17         // ===== DECORATOR =====
18         strategy = new DiscountDecorator(strategy);
19         strategy = new ProcessingFeeDecorator(strategy);
20
21         context.setStrategy(strategy);
22
23         context.processPayment( amount: 1000);
24
25         // chuyển trạng thái
26         context.setState(new CompletedPaymentState());
27         context.processPayment( amount: 1000);
28     }
29 }
  
```

Kết quả

```
Run Main x
"\"C:\\Program Files\\Java\\jdk-21\\bin\\java.exe\" \"-javaagent:C:\\Program Files\\JetBrain
Trạng thái mới tạo: Đang chờ chọn phương thức
Phí xử lý: 20.0
Giảm giá: 102.0
Thanh toán bằng thẻ tín dụng: 918.0
Trạng thái hoàn tất: Giao dịch thành công
Phí xử lý: 20.0
Giảm giá: 102.0
Thanh toán bằng thẻ tín dụng: 918.0

Process finished with exit code 0
```

Luồng chạy:

Lần 1:

Trạng thái mới tạo: Đang chờ chọn phương thức

Giảm giá: 100

Phí xử lý: 18

Thanh toán bằng thẻ tín dụng: 918

Lần 2:

Trạng thái hoàn tất: Giao dịch thành công

Giảm giá: 100

Phí xử lý: 18

Thanh toán bằng thẻ tín dụng: 918

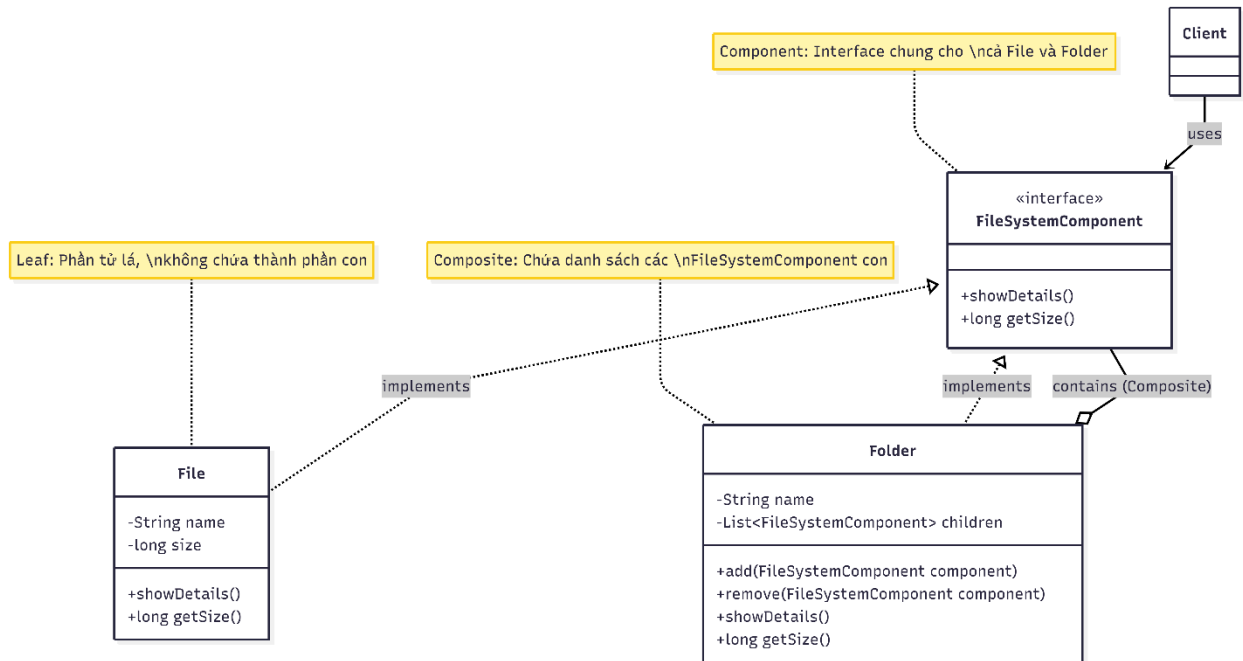
3. Kết luận

- **Mục đích:** Mở rộng tính năng cho đối tượng một cách **động** mà không cần dùng kế thừa.
- **Cơ chế:** "Bọc" đối tượng gốc bằng các lớp trang trí để thêm trách nhiệm (như phí dịch vụ, gói quà) tại thời điểm chạy.

- **Ứng dụng:** Thêm các tùy chọn phụ phí cho hóa đơn hoặc tính năng bổ sung cho sản phẩm.

Bài 4: Composite

1. Sơ đồ design



2. Cấu trúc mã nguồn

The screenshot shows the source code for the Composite Design Pattern in an IDE. The project structure on the left includes **composite-file-system** with subdirectories **src** (containing **File**, **FileSystemComponent**, and **Folder**) and **main** (containing **Main**). The **Main.java** file is open, showing the following code:

```

1 public class Main {
2     public static void main(String[] args) {
3
4         // ===== File =====
5         File file1 = new File( name: "file1.txt", size: 100);
6         File file2 = new File( name: "file2.txt", size: 200);
7         File file3 = new File( name: "file3.txt", size: 300);
8
9         // ===== Folder con =====
10        Folder subFolder = new Folder( name: "SubFolder");
11        subFolder.add(file2);
12        subFolder.add(file3);
13
14        // ===== Folder chính =====
15        Folder root = new Folder( name: "Root");
16        root.add(file1);
17        root.add(subFolder);
18
19        // ===== Hiển thị =====
20        root.showDetails();
21
22        // ===== Tổng size =====
23        System.out.println("Total size: " + root.getSize());
24    }
25 }
  
```

```
Run Main x
"\"C:\\Program Files\\Java\\jdk-21\\bin\\java.exe\" \"-javaagent:C:\\Program Files\\J...
Folder: Root
File: file1.txt | Size: 100
Folder: SubFolder
File: file2.txt | Size: 200
File: file3.txt | Size: 300
Total size: 600

Process finished with exit code 0
```

Luồng hoạt động

- File → Leaf → không chứa gì
- Folder → Composite → chứa List<FileSystemComponent
- showDetails() → gọi đệ quy → in cây
- getSize() → cộng dồn đệ quy

```
root
├─ gọi file1.showDetails()
└─ gọi subFolder.showDetails()
    ├─ file2.showDetails()
    └─ file3.showDetails()
```

3. Kết luận

- Hệ thống sử dụng Composite Pattern để biểu diễn cấu trúc cây giữa File và Folder.
- Folder có thể chứa nhiều File hoặc Folder khác, và thực hiện các thao tác bằng cách gọi đệ quy xuống các phần tử con.
- Nhờ interface chung FileSystemComponent, client có thể xử lý đồng nhất mà không cần phân biệt đối tượng cụ thể.
- **Mục đích:** Xây dựng cấu trúc dữ liệu dạng **cây (tree)** để quản lý các đối tượng phân cấp.

- **Cơ chế:** Giúp Client đối xử với đối tượng đơn lẻ (Leaf) và nhóm đối tượng (Composite) một cách **đồng nhất** thông qua interface chung.
- **Ứng dụng:** Quản lý Folder/File, cấu trúc menu hoặc các thành phần giao diện người dùng (UI components).

Bài 5: Observer Design Pattern

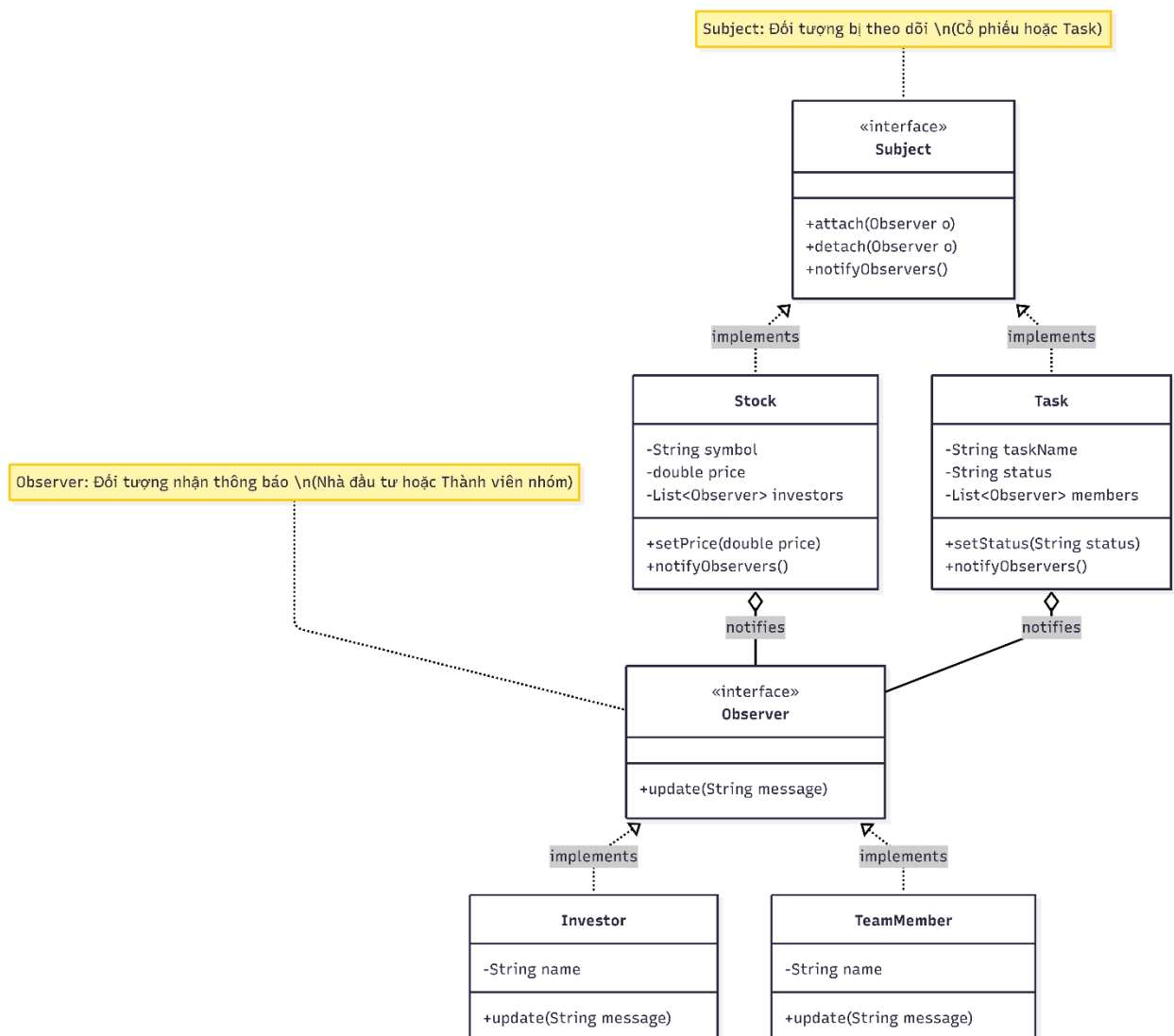
Khi giá của một cổ phiếu thay đổi, các nhà đầu tư đã đăng ký để theo dõi cổ phiếu đó sẽ nhận thông báo ngay lập tức về sự thay đổi.

Trong một dự án phần mềm, khi có sự thay đổi về tình trạng hoặc trạng thái công việc (task), các thành viên trong nhóm sẽ nhận được thông báo tự động để theo dõi tiến độ.

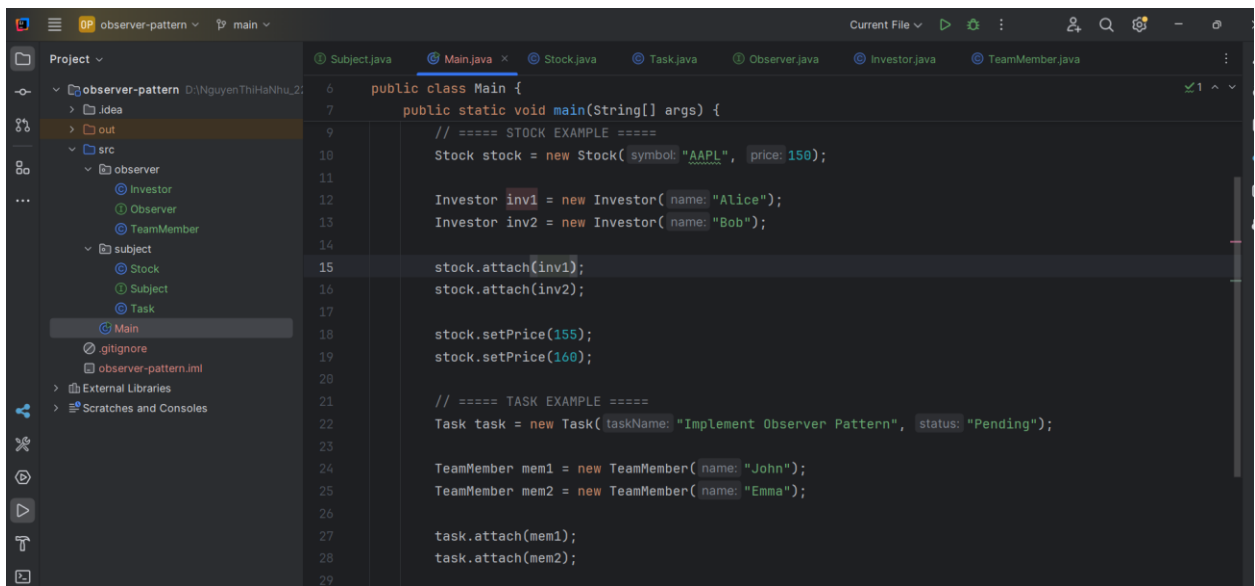
Hãy áp dụng Observer Design Pattern vào các trường hợp trên

Yêu cầu: vẽ sơ đồ trước khi viết code.

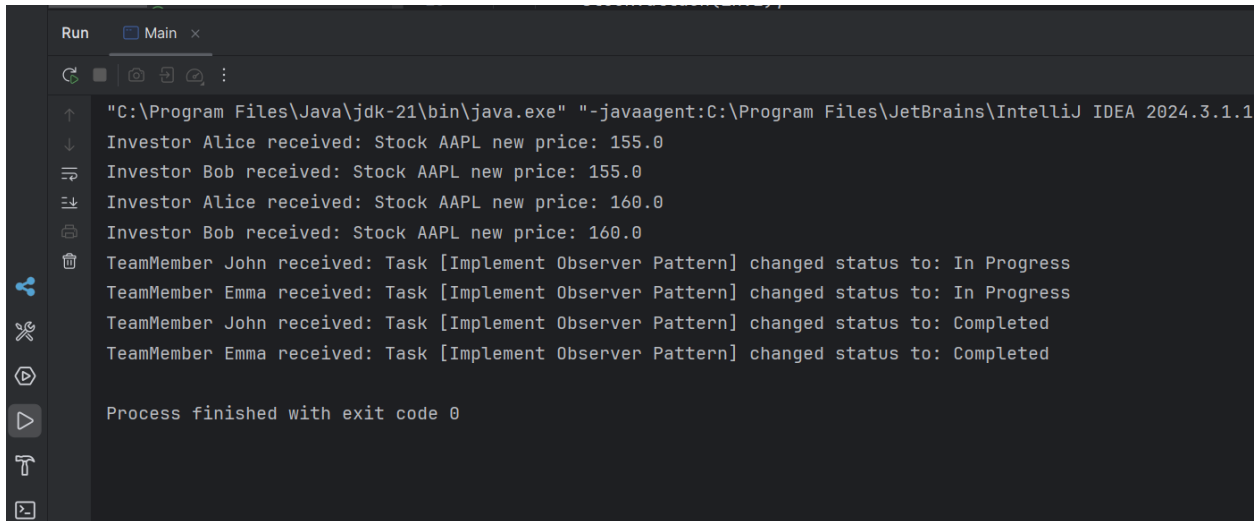
1. Sơ đồ Design



2. Cấu trúc mã nguồn



```
6 public class Main {
7     public static void main(String[] args) {
8         // ===== STOCK EXAMPLE =====
9         Stock stock = new Stock(symbol: "AAPL", price: 150);
10
11         Investor inv1 = new Investor(name: "Alice");
12         Investor inv2 = new Investor(name: "Bob");
13
14         stock.attach(inv1);
15         stock.attach(inv2);
16
17         stock.setPrice(155);
18         stock.setPrice(160);
19
20         // ===== TASK EXAMPLE =====
21         Task task = new Task(taskName: "Implement Observer Pattern", status: "Pending");
22
23         TeamMember mem1 = new TeamMember(name: "John");
24         TeamMember mem2 = new TeamMember(name: "Emma");
25
26         task.attach(mem1);
27         task.attach(mem2);
28
29     }
```



```
Run Main x
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1
Investor Alice received: Stock AAPL new price: 155.0
Investor Bob received: Stock AAPL new price: 155.0
Investor Alice received: Stock AAPL new price: 160.0
Investor Bob received: Stock AAPL new price: 160.0
TeamMember John received: Task [Implement Observer Pattern] changed status to: In Progress
TeamMember Emma received: Task [Implement Observer Pattern] changed status to: In Progress
TeamMember John received: Task [Implement Observer Pattern] changed status to: Completed
TeamMember Emma received: Task [Implement Observer Pattern] changed status to: Completed

Process finished with exit code 0
```

Luồng hoạt động

- Tạo các subject là Stock
 - Tạo observers là Investor gồm Alice và Bob
 - Đăng ký theo dõi Stock cho Alice và Bob thông qua phương thức attach để nhận được thông báo khi cổ phiếu thay đổi giá
 - Tương tự cho bài toán task
- ### 3. Kết luận
- Giúp thông báo được các task cho nhiều đối tượng cùng lúc khi có đăng ký
 - Áp dụng cho các bài toán nhận được update từ 1 thay đổi trạng thái nào đó trong hệ thống
 - **Mục đích:** Thiết lập mối quan hệ **1-nhiều**, giúp cập nhật dữ liệu đồng bộ giữa các đối tượng.

- **Cơ chế:** Khi đối tượng chính (Subject) thay đổi trạng thái, tất cả các đối tượng đăng ký theo dõi (Observers) sẽ tự động nhận thông báo.
- **Ứng dụng:** Hệ thống thông báo chứng khoán, thông báo tiến độ công việc hoặc sự kiện trong UI.

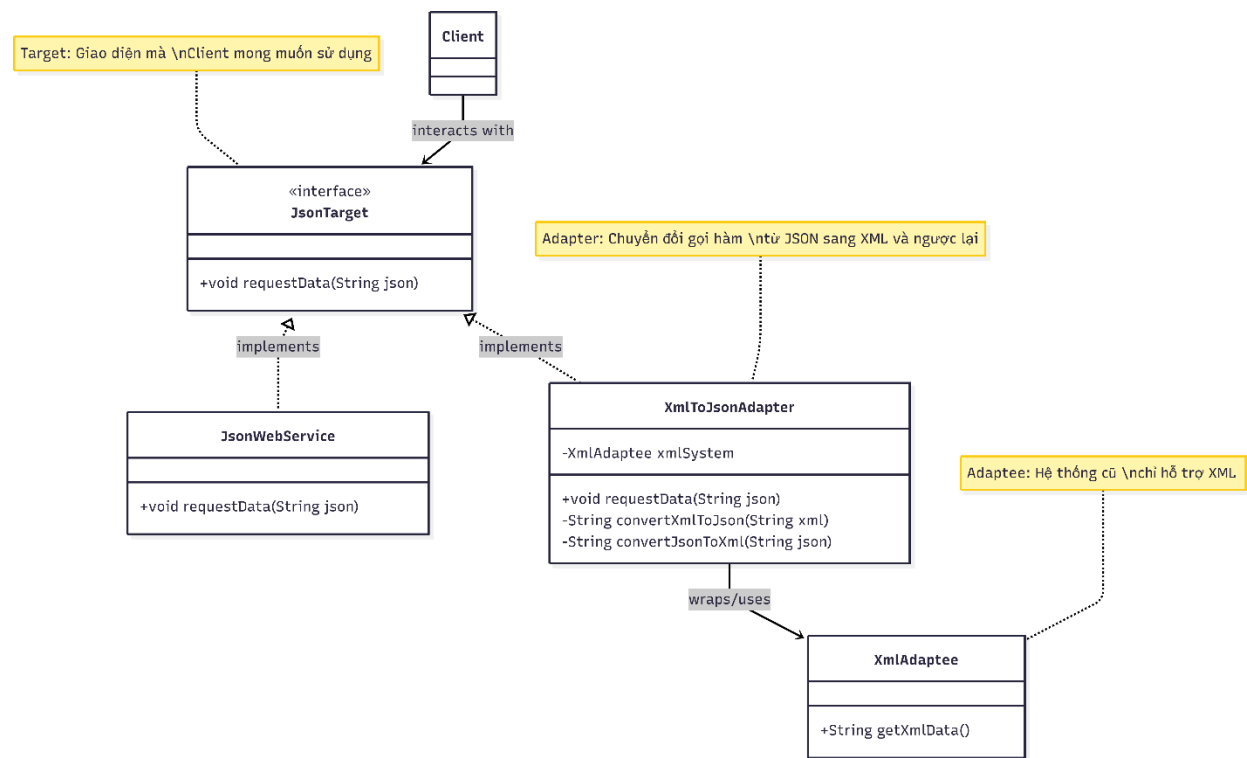
Bài 6 : Adapter Pattern

Một dịch vụ web yêu cầu đầu vào ở định dạng JSON, nhưng một hệ thống khác chỉ hỗ trợ XML. Bạn có thể viết một adapter để chuyển đổi dữ liệu từ XML sang JSON và ngược lại.

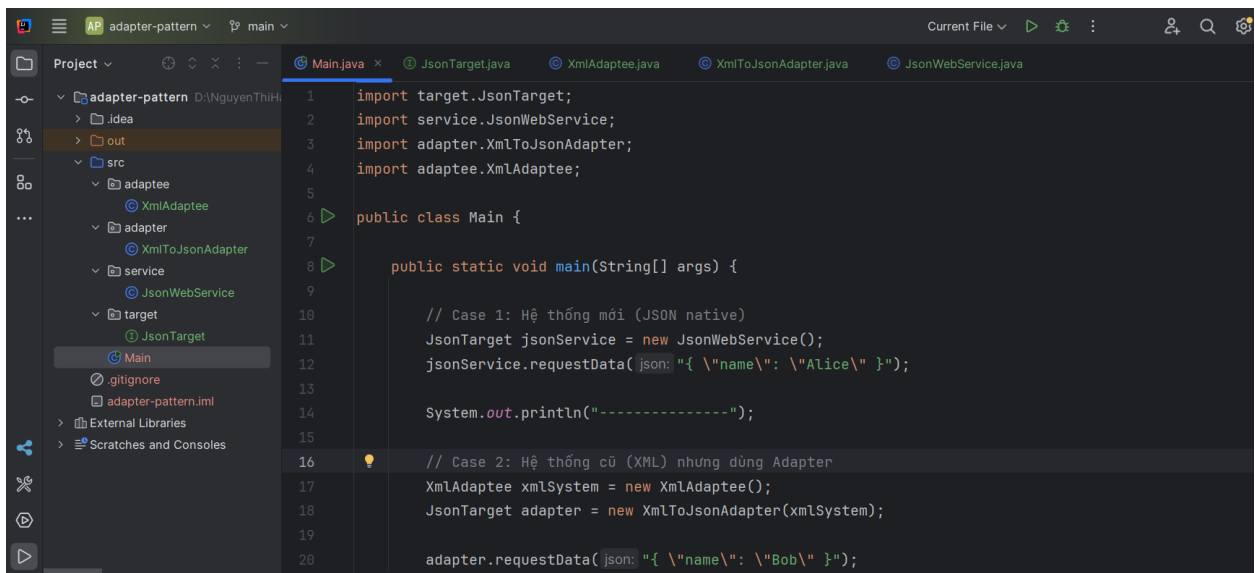
Hãy áp dụng Adapter Design Pattern vào trường hợp trên

Yêu cầu: vẽ sơ đồ trước khi viết code.

1. Sơ đồ design

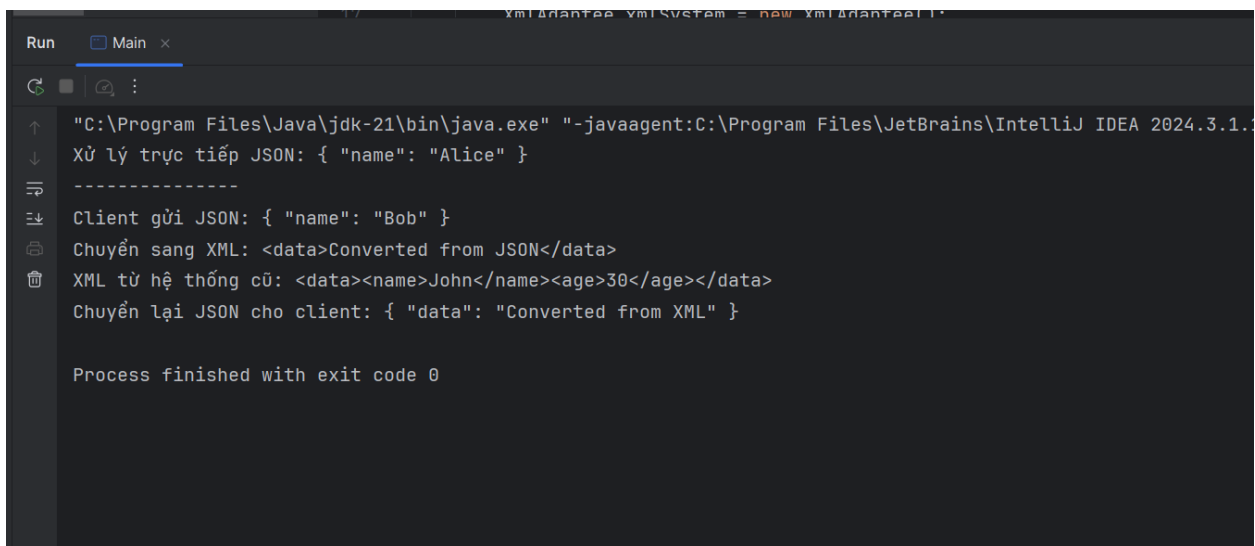


2. Cấu trúc mã nguồn



The screenshot shows the IntelliJ IDEA interface with the 'adapter-pattern' project. The Project Explorer on the left shows the package structure: adapter-pattern, .idea, out, src, adapter, adapter.adaptee, adapter.XmlToJsonAdapter, service, service.JsonWebService, target, target.JsonTarget, Main, .gitignore, adapter-pattern.iml, External Libraries, and Scratches and Consoles. The Main.java file is open in the editor, showing the following code:

```
1 import target.JsonTarget;
2 import service.JsonWebService;
3 import adapter.XmlToJsonAdapter;
4 import adaptee.XmlAdaptee;
5
6 public class Main {
7
8     public static void main(String[] args) {
9
10         // Case 1: Hệ thống mới (JSON native)
11         JsonTarget jsonService = new JsonWebService();
12         jsonService.requestData(json: "{ \"name\": \"Alice\" }");
13
14         System.out.println("-----");
15
16         // Case 2: Hệ thống cũ (XML) nhưng dùng Adapter
17         XmlAdaptee xmlSystem = new XmlAdaptee();
18         JsonTarget adapter = new XmlToJsonAdapter(xmlSystem);
19
20         adapter.requestData(json: "{ \"name\": \"Bob\" }");
```

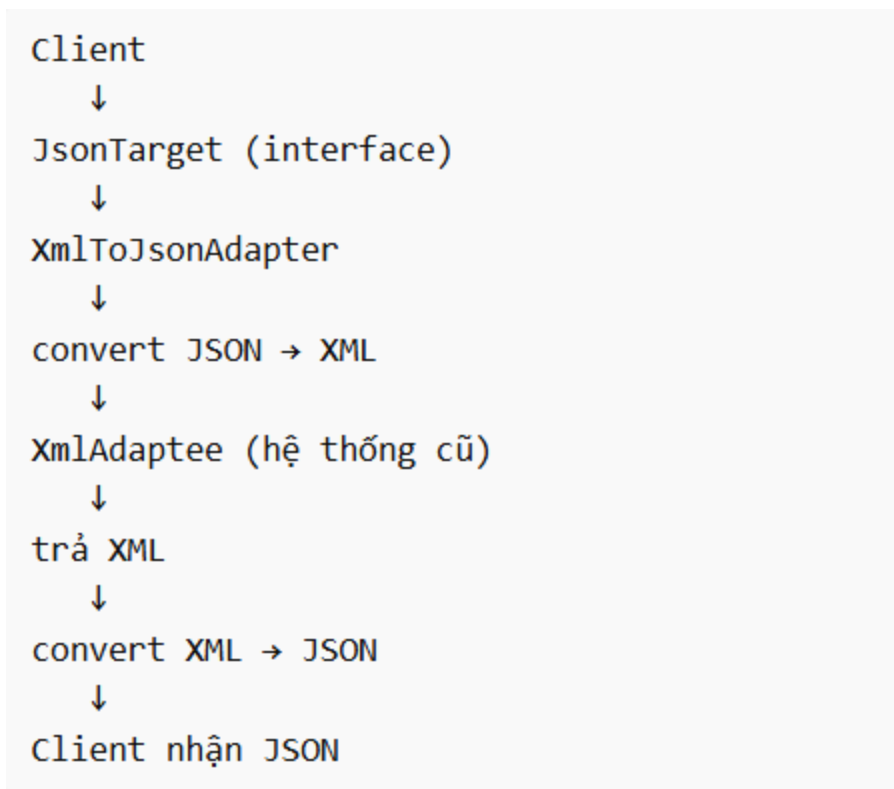


The screenshot shows the Run console in IntelliJ IDEA. The console output is as follows:

```
Run Main x
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\lib\idea_rt.jar=60253:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\bin" -Dfile.encoding=UTF-8
Xử lý trực tiếp JSON: { "name": "Alice" }
-----
Client gửi JSON: { "name": "Bob" }
Chuyển sang XML: <data>Converted from JSON</data>
XML từ hệ thống cũ: <data><name>John</name><age>30</age></data>
Chuyển lại JSON cho client: { "data": "Converted from XML" }

Process finished with exit code 0
```

Luồng hoạt động

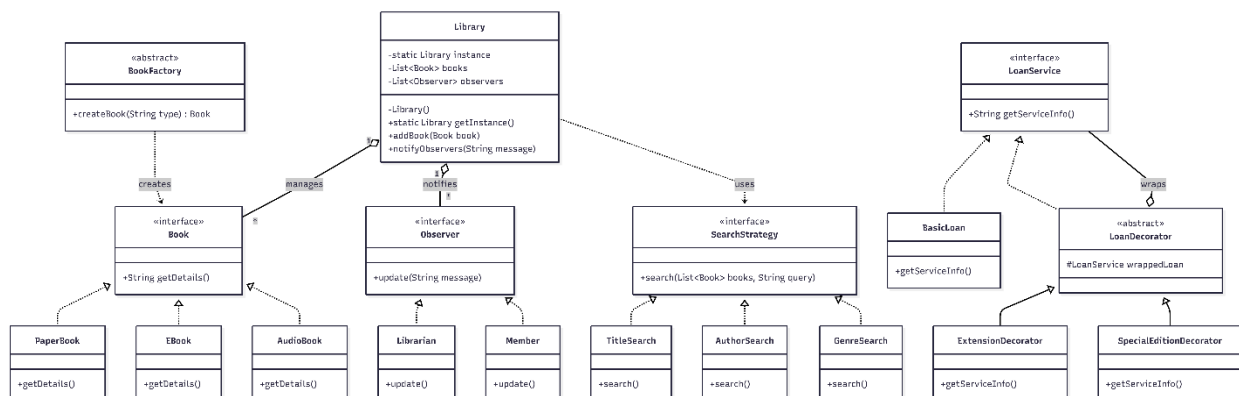


3. Kết luận

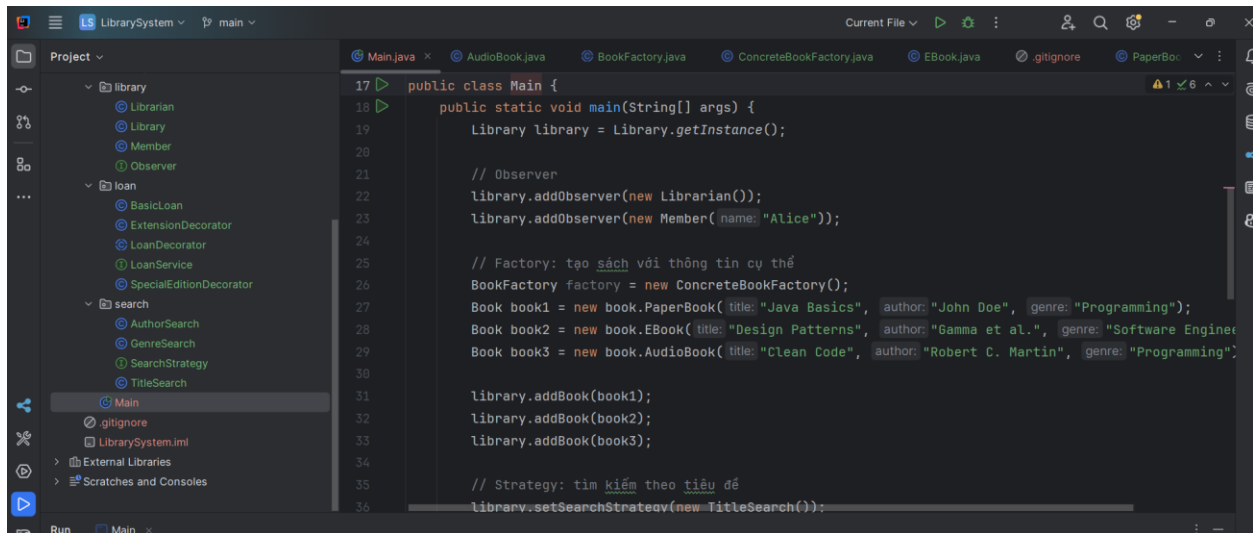
- **Mục đích:** Giải quyết sự **không tương thích** về giao diện giữa Client và hệ thống sẵn có (Adaptee).
- **Cơ chế:** Đóng vai trò lớp trung gian để chuyển đổi dữ liệu/gọi hàm mà không cần sửa đổi mã nguồn gốc.
- **Ứng dụng:** Kết nối các hệ thống khác biệt (như XML và JSON) hoặc tích hợp thư viện cũ vào dự án mới.

Bài Tổng Hợp 6 pattern: Quản lý thư viện

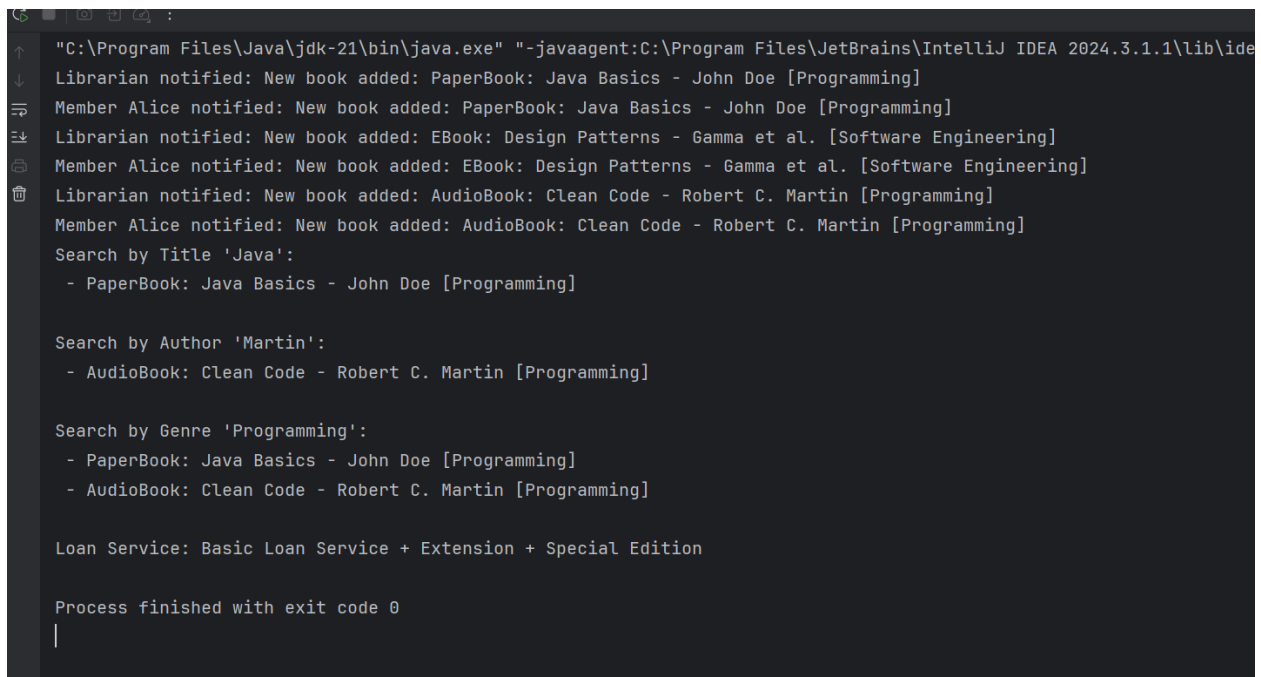
1. Sơ đồ Design



2. Cấu trúc mã nguồn



```
17 public class Main {
18     public static void main(String[] args) {
19         Library library = Library.getInstance();
20
21         // Observer
22         library.addObserver(new Librarian());
23         library.addObserver(new Member(name: "Alice"));
24
25         // Factory: tạo sách với thông tin cụ thể
26         BookFactory factory = new ConcreteBookFactory();
27         Book book1 = new book.PaperBook(title: "Java Basics", author: "John Doe", genre: "Programming");
28         Book book2 = new book.EBook(title: "Design Patterns", author: "Gamma et al.", genre: "Software Engineering");
29         Book book3 = new book.AudioBook(title: "Clean Code", author: "Robert C. Martin", genre: "Programming");
30
31         library.addBook(book1);
32         library.addBook(book2);
33         library.addBook(book3);
34
35         // Strategy: tìm kiếm theo tiêu đề
36         library.setSearchStrategy(new TitleSearch());
```



```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\lib\ide
Librarian notified: New book added: PaperBook: Java Basics - John Doe [Programming]
Member Alice notified: New book added: PaperBook: Java Basics - John Doe [Programming]
Librarian notified: New book added: EBook: Design Patterns - Gamma et al. [Software Engineering]
Member Alice notified: New book added: EBook: Design Patterns - Gamma et al. [Software Engineering]
Librarian notified: New book added: AudioBook: Clean Code - Robert C. Martin [Programming]
Member Alice notified: New book added: AudioBook: Clean Code - Robert C. Martin [Programming]
Search by Title 'Java':
- PaperBook: Java Basics - John Doe [Programming]

Search by Author 'Martin':
- AudioBook: Clean Code - Robert C. Martin [Programming]

Search by Genre 'Programming':
- PaperBook: Java Basics - John Doe [Programming]
- AudioBook: Clean Code - Robert C. Martin [Programming]

Loan Service: Basic Loan Service + Extension + Special Edition

Process finished with exit code 0
|
```

- **book/**: Chứa các lớp liên quan đến sách và Factory Method.
- **library/**: Chứa lớp Library (Singleton) và các lớp Observer (Librarian, Member).
- **search/**: Chứa các chiến lược tìm kiếm (Strategy Pattern).
- **loan/**: Chứa các lớp liên quan đến dịch vụ mượn sách và Decorator Pattern.
- **Main.java**: Chạy demo toàn hệ thống.

3. Kết luận

Trong hệ thống quản lý thư viện mà bạn đã xây dựng, mỗi **Design Pattern** đóng một vai trò rõ ràng và giải quyết một vấn đề cụ thể:

1. Singleton Pattern (Library)

- Đảm bảo chỉ có một đối tượng Library duy nhất trong toàn hệ thống.
- Giúp quản lý tập trung danh sách sách và danh sách Observer.
- Tránh việc tạo nhiều thư viện khác nhau gây mất đồng bộ dữ liệu.

2. Factory Method Pattern (BookFactory)

- Cho phép tạo ra nhiều loại sách khác nhau (PaperBook, EBook, AudioBook) mà không cần thay đổi code ở phía client.
- Giúp hệ thống dễ mở rộng khi thêm loại sách mới (ví dụ: BrailleBook, ComicBook).
- Tách biệt logic khởi tạo khỏi phần sử dụng.

3. Strategy Pattern (SearchStrategy)

- Cho phép thay đổi thuật toán tìm kiếm linh hoạt mà không ảnh hưởng đến lớp Library.
- Người dùng có thể chọn tìm theo **tiêu đề**, **tác giả**, hoặc **thể loại**.
- Dễ mở rộng thêm các chiến lược khác (ví dụ: tìm theo năm xuất bản, ISBN).

4. Observer Pattern (Librarian, Member)

- Khi có sự kiện (thêm sách mới, sách hết hạn mượn), hệ thống tự động thông báo cho những người quan tâm.
- Giúp tách biệt logic quản lý sách với logic thông báo.
- Dễ mở rộng thêm nhiều loại Observer khác nhau (ví dụ: hệ thống email, ứng dụng di động).

5. Decorator Pattern (LoanService)

- Cho phép mở rộng dịch vụ mượn sách mà không cần thay đổi lớp gốc.
- Ví dụ: thêm tính năng **gia hạn thời gian mượn**, **phiên bản đặc biệt**.
- Dễ dàng kết hợp nhiều tính năng bổ sung bằng cách bọc chồng các Decorator.